

The *.gitignore* file that the `cookiecutter-basicpythonproject` template creates looks like this:

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[cod]
*$py.class
--snip--
```

The *.gitignore* file uses `*` for wildcards and `#` for comments. You can read more about it in the online documentation at <https://git-scm.com/docs/gitignore>.

You should add the actual *.gitignore* file to the Git repo so other programmers have it if they clone your repo. If you want to see which files in your working directory are being ignored based on the settings in *.gitignore*, run the `git ls-files --other --ignored --exclude-standard` command.

Committing Changes

After adding new files to the repo, you can continue writing code for your project. When you want to create another snapshot, you can run `git add .` to stage all modified files and `git commit -m <commit message>` to commit all staged files. But doing so is easier with the single `git commit -am <commit message>` command:

```
C:\Users\Al\wizcoin>git commit -am "Fixed the currency conversion bug."
[master (root-commit) e1ae3a3] Fixed the currency conversion bug.
1 file changed, 12 insertions(+)
```

If you want to commit only certain modified files instead of every modified file, you can omit the `-a` option from `-am` and specify the files after the commit message, such as `git commit -m <commit message> file1.py file2.py`.

The commit message provides a hint for future use: it's a reminder about what changes we made in this commit. It might be tempting to write a short, generic message, such as "Updated code," or "Fixed a few bugs," or even just "x" (because blank commit messages aren't allowed). But three weeks from now, when you need to roll back to an earlier version of your code,